MongoDB Sharding Guide

Sharding is a method for distributing data across multiple machines. MongoDB uses sharding to support deployments with very large data sets and for high throughput operations.

Scaling

Database systems with large data sets or high throughput applications can challenge the capacity of a single server. For example, high query rates can exhaust the CPU capacity of the server. Working set sizes larger than the system's RAM stress the I/O capacity of disk drives.

These issues can be overcome by scaling the systems. There are 2 methods for doing this.

- **Vertical Scaling Scaling**

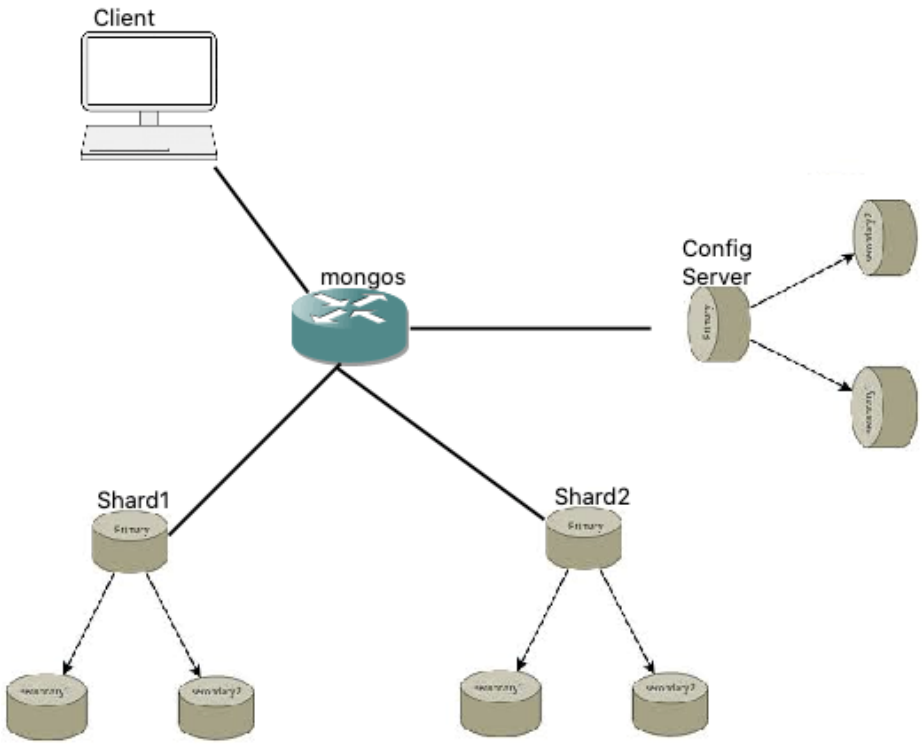
Vertical Scaling involves increasing the capacity of a single server, such as using a more powerful CPU, adding more RAM, or increasing the amount of storage space.

- **Horizontal Scaling**

Horizontal Scaling involves dividing the system dataset and load over multiple servers, adding additional servers to increase capacity as required. While the overall speed or capacity of a single machine may not be high, each machine handles a subset of the overall workload, potentially providing better efficiency than a single high-speed high-capacity server.

MongoDB supports horizontal scaling through sharding.

MongoDB Sharding Architecture



Sharding Components

Shard

Each shard contains a subset of the sharded data (Collection). Each shard can be deployed as a mogodb replica set.

mongos

The mongos acts as a query router, providing an interface between client applications and the sharded cluster. The mongos instances send pings to all members of the cluster every 30 seconds so the cluster can verify that the mongos is active.

config servers

Config servers store metadata and configuration settings for the cluster. Config servers must be deployed as a replica set (CSRS). Config Servers does not saves any of the user data.

Shard Keys

MongoDB uses the shard key to distribute the collection's documents across shards. The shard key consists of a field or fields that must exist in every document in the target collection and it should be indexed preferably hash index.

Not any of the databases or collections are sharded until user choose to shard them.

Following steps should be done for sharding a collection.

1. Enable sharding on database
2. Select a field which becomes shard key. (this field must exists in all documents in the collections)
3. Index the key
4. Shard the collections

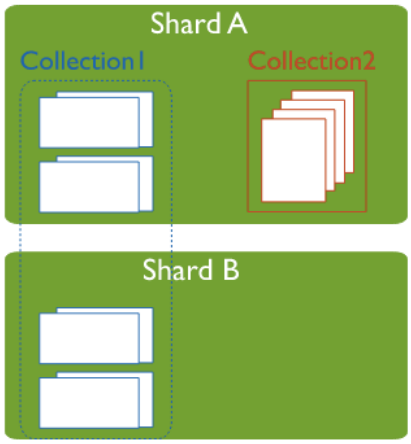
Chunks

MongoDB partitions sharded data into chunks (128MB default). Each chunk has an inclusive lower and exclusive upper range based on the shard key.

In an attempt to achieve an even distribution of chunks across all shards in the cluster, a balancer runs in the background to migrate chunks across the shards.

Sharded and Non-Sharded Collections

A database can have a mixture of sharded and unsharded collections. Sharded collections are partitioned and distributed across the shards in the cluster. Unsharded collections are stored on



a primary shard. Each database has its own primary shard.

Setting up mongoDB Sharding

Setting up MongoDB sharding consists of deploying the following components

- Config Servers Replica Set
- Mongos Server
- Individual MongoDB Replica Sets which hold databases and collections

Note: Same key file must be used across all replica sets and config servers replica set to enable seamless communication among them.

Setting-Up Config Servers Replica Set

Config Servers uses mongod process as normal mongoDB servers but in the configuration file clusterRole should be set to configsvr.

A sample config file below

```
sharding:
  clusterRole: configsvr
replication:
  replSetName: con-sers
security:
  keyFile: /var/mongodb/pki/mc-keyfile
net:
  bindIp: localhost,192.168.103.100
  port: 26001
systemLog:
  destination: file
  path: /var/mongodb/db/csrs1.log
  logAppend: true
processManagement:
  fork: true
storage:
  dbPath: /var/mongodb/db/csrs1
```

The Procedure to setup and manage the Config Servers is same as any normal MongoDB Replica Set.

1. Create a Configuration file for each Config Server in Replica Set.
2. Create a key file for Authentication
3. Create all required Directories and set permissions
4. Start each of the mongod servers
5. Login to first node and initiate cluster and create super user
6. Add other config Servers

Starting The Config Servers

```
mongod -f csrs_1.conf
mongod -f csrs_2.conf
mongod -f csrs_3.conf
```

Login into first node and initiate replica set

```
mongo --port 26001

rs.initiate(
 {
    _id: "ctrls-cfg",
    configsvr: true,
    members: [
      { _id : 0, host : "training:27024" },
      { _id : 1, host : "training:27025" },
      { _id : 2, host : "training:27026" }
    ]
  }
)
```

Add Super User

```
use admin
db.createUser({
  user: "admin",
  pwd: "pass12345",
  roles: [
    {role: "root", db: "admin"}
  ]
})
```

Authenticating as the super user:

```
db.auth("admin", "pass12345")
```

That should complete the Configuration Server Replica Set setup.

Setup Mongos Server

Mongos config (mongos.conf):

```
sharding:
  configDB: con-
sers/192.168.103.100:26001,192.168.103.100:26002,192.168.103.100:26003
security:
  keyFile: /var/mongodb/pki/m103-keyfile
net:
  bindIp: 0.0.0.0
  port: 26000
systemLog:
  destination: file
  path: /var/log/mongod/mongos.log
  logAppend: true
processManagement:
  fork: true
```

Starting mongos Server

```
mongos -f  /etc/mongos.conf
```

Connecting to Mongos

```
mongo --port 26000 --username admin --password pass12345 --
authenticationDatabase admin
```

Checking Sharding Status

```
sh.status()
```

Mongos does not have its own authentication database but instead relies on config servers for authentication information.

Adding MongoDB Replica Sets to Sharded Cluster

Add the following lines to the configuration file of each node of replica set.

```
sharding:
  clusterRole: shardsvr
```

Perform a rolling restart on replica set.

Rolling Restart:

1. Restart one of the secondary
2. Restart other secondary
3. Once all secondaries are restarted, login to Primary and rs.stepDown()
4. Restart

Once Restart is completed and sharding configuration is applied to the replica set login to mongos and add the shard

```
mongos --port 26000 --username admin --password pass12345 --
authenticationDatabase admin
```

```
sh.addShard("mc-repl/<primary IP of replica set>:12345")
```

Sharding a Collection

Enabling sharding on a database.

Connect to mongos to enable sharding on a database. By enabling sharding on a database we can shard a collection belonging to that database.

```
sh.enableSharding("<database name>")
```

Shard a Collection

First of all select a field which can be used as shard key on the collection. Make sure that key is present in all the documents and it is indexed, if not indexed then index it.

Run the following command to shard the collection

```
db.adminCommand( { shardCollection: "databasename.collection", key: {
<shard_key>: 1 } } )
```

Cluster Configuration and Administration

The Config Database

Config database is used by MongoDB internally to store cluster information and state. Config database contain several collection to hold cluster wide configuration and status information.

WARNING: Modification of the config database on a functioning system may lead to instability or inconsistent data sets.

The Following collections are maintained by MongoDB to support sharding

config.changelog

The changelog collection stores a document for each change to the metadata of a sharded collection.

config.chunks

The chunks collection stores a document for each chunk in the cluster.

`config.collections`

The collections collection stores a document for each sharded collection in the cluster.

`config.databases`

The databases collection stores a document for each database in the cluster.

For each database, the corresponding document displays the name, the database's primary shard, the database's sharding enabled status, and a version.

`config.lockpings`

The lockpings collection keeps track of the active components in the sharded cluster.

`config.locks`

The locks collection stores the distributed locks. The primary of the config server replica set takes a lock by inserting a document into the locks collection.

`config.mongos`

The mongos collection stores a document for each mongos instance affiliated with the cluster.

`config.settings`

The settings collection holds the configuration settings of the cluster.

`config.shards`

The shards collection represents each shard in the cluster in a separate document

`config.tags`

The tags collection holds documents for each zone range in the cluster.

`config.version`

The version collection holds the current metadata version number. This collection contains only one document.

`config.system.sessions`

The system.sessions collection stores session records that are available to all members of the deployment.

When a user creates a session on a mongod or mongos instance, the record of the session initially exists only in-memory on the instance. Periodically, the instance will sync its cached sessions to the system.sessions collection; at which time, they are visible to all members of the deployment.

`config.transactions`

The transactions collection stores records used to support retryable writes for replica sets and sharded clusters.

Managing Balancer

Balancer is responsible for equal distribution of data among the shards for which balancer constantly moves the chunks from one shard to another. The behaviors of balancer can be managed by following commands.

Start / Stop Balancer

Connect to mongos and run the following command.

```
sh.startBalancer()
```

Starts / enables the balancer.

```
sh.stopBalancer()
```

If the balancer is stopped moving of chunks around the shards also stop and over period of time there will huge uneven distribution of data which will result in degraded performance of the cluster.

Check Balancer State

```
sh.getBalancerState()
```

Checks is the balancer is enabled or not

```
sh.isBalancerRunning()
```

Checks if the balancer is running

Setting Balancer runtime Window

On a busy cluster migrating chunks may result in performance issues thus it is needed to be disabled (stopped) during certain time. This can be achieved by adding a activeWindow field to balancer document (_id: "balancer") in the settings collection of config database. The following code will be useful

Note: before configuring time window make is sure balancer is enabled.

```
use config
db.settings.update(
   { _id: "balancer" },
   { $set: { activeWindow : { start : "<HH:MM>", stop : "<HH:MM>" } }
},
   { upsert: true }
)
```

Where HH is hours in 24 hour format (0-23) and MM is Minutes (0-59).

Unset Balancer runtime window

Run the following code to unset the balancer window

```
use config
db.settings.update({ _id : "balancer" }, { $unset : { activeWindow :
true } })
```

Viewing Cluster Configuration

List Databases with Sharding Enabled

```
use config
db.databases.find( { "partitioned": true } )
```

List all the shards

```
db.adminCommand( { listShards : 1 } )
```

View Cluster Details

```
db.printShardingStatus()
sh.status()
```

Restarting Sharded Custer

- Disable the Balancer

  Login to mongos using mongo shell and run following command

  ```
  sh.stopBalancer()
  ```

- Stop Sharded Cluster
  - Stop mongos routers (all of them)

    ```
    use admin
    db.shutdownServer()
    ```

  - Stop each shard replica set

    Run the following command on the each member of the Shards replica set. Repeat this for all shards.

    ```
     use admin
    db.shutdownServer()
    ```

  - Stop config servers

    Run the following command on the each member of the config servers replica set.

    ```
    use admin
    db.shutdownServer()
    ```

- Start Sharded Cluster
  - Start config servers

    Run any one of the following command on each of the servers in config server replica set

```
mongod --config <path-to-config-file>
systemctl  start mongod
```

- Start each shard replica set

  Run any one of the following command on each of the servers in shard's replica set. Repeat same for all shards

```
mongod --config <path-to-config-file>
systemctl  start mongod
```

- Start mongos routers

  Run any one of the following command on each of the mongos router servers. Repeat same for all mongos routers

```
mongos --config <path-to-config-file>
systemctl  start mongos
```

- Re-Enable the Balancer

  Login to mongos using mongo shell and run the following command

```
sh.startBalancer()
```

- Validate Cluster Accessibility

```
Check if the clients and application is able connect
```

Adding Shards

Any time additional shard can be added to cluster and there is no limit on number of shards which can be added to cluster.

The following procedure adds a shard to cluster

First configure the replica set to be added

Connect to mongos using mongo shell and run the following commands

```
sh.addShard( "<replica set name>/<primary's host name in replica set>:<port>" )
```

Removing Shards

Before removing a shard ensure that the balancer is enabled and if it is not then first enable balancer.

Process of shard migration can take few minutes to complete to several days depending the size of data.

The following command starts shard migration

```
db.adminCommand( { removeShard: "<shard name>" } )
```

Any time if you want to know the status of migration issue the same command once again.

```
db.adminCommand( { removeShard: "<shard name>" } )
```

For list of shards names run the following command

```
db.adminCommand( { listShards: 1 } )
```

Cluster Backups

Before any backup activity on the cluster balancer must be stopped.

If the balancer is enabled it will be always moving the chunks around the shards and the backups running during this time may end up inconsistent.

Cluster backups must be started at same time across all shards to ensure data consistency. For better time coordination configure all nodes in cluster to use same time server. Also automate backups using cron jobs and avoid manual backups as there will be time difference.

Chunks Size

The default chunk size for a sharded cluster is 128 megabytes. This default chunk size works well for most deployments. A lower chunk size result in less I/O and more frequent chunk migration. If chunk migrations are causing lot of I/O than your hardware can handle, you may want to reduce the chunk size.

Issue the following save operation on the config db's settings collection to modify the chunk size

```
use config
db.settings.save( { _id:"chunksize", value: <sizeInMB> } )
```

Allowed size range is 1 to 1024 MB.

- If you lower the chunk size, it may take time for all chunks to split to the new size.
- If you increase the chunk size, existing chunks grow only through insertion or updates until they reach the new size.